

Cómo construir una pasarela de pagos crypto propia

Guía técnica: recepción de depósitos en USDT (TRC-20) mediante direcciones únicas por usuario, códigos QR y detección automática de transacciones.

Este documento explica, de forma general y aplicable a cualquier proyecto, cómo diseñar un sistema propio para recibir pagos en criptomonedas (USDT sobre la red TRON/TRC-20 como ejemplo) sin depender de un procesador de pago tradicional. Es el mismo enfoque que usan exchanges como Binance: cada usuario recibe una dirección de depósito única, generada matemáticamente a partir de una sola "semilla maestra" (HD Wallet), lo que permite identificar automáticamente quién envió cada pago sin necesidad de un banco o pasarela externa.

Nota: esto no es asesoría legal ni financiera. Antes de mover dinero real de terceros, cualquier proyecto debe validar con un abogado los requisitos regulatorios aplicables en su jurisdicción (KYC/AML, licencias de operador de pagos o dinero electrónico, retención de logs, etc.). Este documento cubre únicamente la arquitectura técnica.

Visión general del flujo

```
Usuario se registra en la plataforma
|
v
Backend genera una direccion TRC-20 EXCLUSIVA para el
(derivada de un indice unico, ej. su ID incremental)
|
v
Se guarda en BD: usuario_1 -> TQn8i4...AA01
|
v
Usuario ve SU codigo QR con SU direccion
|
v
Envia USDT desde su wallet (Binance, Trust Wallet, etc.)
|
v
Backend monitorea la blockchain y detecta el deposito
|
v
Busca en BD a que usuario pertenece esa direccion
|
v
Acredita el saldo correspondiente al usuario
```

1. La base: HD Wallet (una semilla, millones de direcciones)

El componente central es una **HD Wallet** (Hierarchical Deterministic Wallet, estándar BIP32/BIP44). A partir de una única frase semilla (mnemonic) se pueden derivar matemáticamente infinitas direcciones distintas, cada una asociada a un índice numérico. La ventaja clave: no necesitas guardar una clave privada por cada usuario — todas se derivan de la misma semilla cuando se necesitan.

```
Seed maestra (guardada cifrada, ej. en un KMS/HSM)
|
+-- usuario 1 (indice 0)  -> TQn8i4...AA01
+-- usuario 2 (indice 1)  -> TQn8i4...BB02
+-- usuario 3 (indice 2)  -> TQn8i4...CC03
+-- usuario N (indice N)  -> TQn8i4...ZZ99
```

Generación de direcciones en código (Node.js + tronweb):

```
import TronWeb from 'tronweb'

// La seed se genera UNA sola vez al crear la wallet maestra.
// Se guarda cifrada (KMS/HSM) - NUNCA en texto plano en el código o BD.
const mnemonic = process.env.MASTER_SEED_MNEMONIC

function generateAddressForUser(userIndex) {
  // Ruta de derivación BIP44 para Tron: m/44'/195'/0'/0/{index}
  const path = `m/44'/195'/0'/0/${userIndex}`
  const account = TronWeb.fromMnemonic(mnemonic, path)
  return {
    address: account.address,      // ej. TQn8i4...AA01
    privateKey: account.privateKey // no se expone nunca al frontend
  }
}

// Al registrarse un usuario nuevo:
const { address } = generateAddressForUser(userIndex)
// INSERT INTO wallets (user_id, derivation_index, address)
// VALUES (1, 0, 'TQn8i4...AA01')
```

Buena práctica: guarda en tu base de datos de aplicación solo la dirección pública y el índice de derivación — nunca la clave privada individual. La clave privada se recalcula al vuelo, únicamente en el momento en que se necesita mover fondos (ver sección 4).

2. Generar el código QR

El QR no es más que una representación visual de la dirección — no requiere ninguna plataforma externa, se genera en tu propio backend (o incluso en el frontend) con una librería estándar.

```
// npm install qrcode
import QRCode from 'qrcode'

const address = 'TQn8i4...AA01' // direccion TRC-20 del usuario

// Genera el QR como imagen base64, lista para
const qrImage = await QRCode.toDataURL(address)
```

El usuario escanea ese QR con cualquier wallet (Binance, Trust Wallet, etc.), ve la dirección pre-llenada y solo escribe el monto a enviar. Opcionalmente, algunos formatos de QR permiten codificar dirección + monto juntos usando un esquema de "payment request" (ej. `tron:TQn8i4...?amount=50`), lo que reduce el error humano si la wallet destino lo soporta.

3. Detectar los depósitos entrantes

Tu backend necesita enterarse cuándo llega dinero a cada una de las miles de direcciones generadas. Hay dos enfoques:

A) Polling (más simple, bueno para un MVP)

Un job programado consulta periódicamente el historial de transacciones de cada dirección contra un nodo/API de la red (ej. TronGrid) y compara con lo ya procesado.

```
import axios from 'axios'

async function checkDeposits() {
  const wallets = await db.query('SELECT user_id, address FROM wallets')

  for (const wallet of wallets) {
    const res = await axios.get(
      `https://api.trongrid.io/v1/accounts/${wallet.address}/transactions/trc20`,
      { params: { contract_address: USDT_TRC20_CONTRACT, limit: 20 } }
    )

    for (const tx of res.data.data) {
      const yaExiste = await db.query(
        'SELECT 1 FROM deposits WHERE tx_hash = ?', [tx.transaction_id]
      )
      if (!yaExiste) {
        await creditarBalance(wallet.user_id, tx.value, tx.transaction_id)
      }
    }
  }
}

// Ejecutar cada 15-30 segundos con un cron/worker
```

B) Webhooks (mejor para escala)

En vez de preguntar constantemente, un proveedor (TronGrid, Tatum, y similares) te notifica automáticamente cuando llega un depósito a cualquiera de las direcciones que monitoreas. Reduce carga en tu backend y baja la latencia de detección, pero requiere configurar el servicio y validar la firma/autenticidad de cada webhook recibido.

4. Acreditar el saldo de forma segura

```

async function creditarBalance(userId, amountUSDT, txHash) {
  await db.transaction(async (trx) => {
    // Idempotencia: evita doble acreditacion si el cron corre 2 veces
    await trx.query(
      'INSERT INTO deposits (user_id, amount, tx_hash, status) VALUES (?, ?, ?, ?)',
      [userId, amountUSDT, txHash, 'confirmed']
    )
    await trx.query(
      'UPDATE users SET balance = balance + ? WHERE id = ?',
      [amountUSDT, userId]
    )
  })
}

```

Puntos críticos en esta etapa:

- **Espera confirmaciones antes de acreditar:** en Tron se recomienda esperar ~19-20 confirmaciones de bloque antes de considerar un depósito "final", para evitar reorganizaciones de cadena (reorgs). No acredites en estado "pending".
- **Idempotencia por hash de transacción:** el tx_hash debe ser único en tu tabla de depósitos — así, si el proceso corre dos veces por error, nunca se acredita el mismo depósito dos veces.
- **Nunca expongas** el derivation_index ni la privateKey en ninguna respuesta de API, log o panel de administración.

5. Barrido de fondos ("sweep") hacia una wallet fría

Las direcciones individuales generadas por usuario normalmente no son el lugar donde "vive" el dinero de forma permanente. Un job programado recorre periódicamente cada dirección hija con saldo y transfiere los fondos hacia una wallet central — idealmente una wallet fría o un esquema multisig (varias firmas requeridas, ej. fundadores + un tercero independiente), en vez de una única clave con acceso total.

- Reduce el riesgo: si una dirección hija individual se ve comprometida, la exposición es mínima porque los fondos ya fueron barridos hacia la wallet fría.
- El esquema multisig evita que una sola persona (founder, admin, developer) tenga control unilateral sobre los fondos de todos los usuarios — un principio que suele ser revisado de cerca por abogados y reguladores de servicios financieros.
- El barrido debe dejar un registro (log) inmutable de cada movimiento: origen, destino, monto, hash de transacción y fecha.

6. Seguridad y cumplimiento — no técnico, pero indispensable

Construir el mecanismo técnico es solo una parte del sistema. Antes de operar con dinero real de terceros, cualquier proyecto normalmente necesita resolver:

- **KYC (Know Your Customer):** verificación de identidad antes de permitir retiros, para vincular cada wallet anónima con una persona real.
- **AML (Anti-Money Laundering):** monitoreo de patrones sospechosos (montos inusuales, frecuencia atípica, múltiples cuentas relacionadas).
- **Licencias aplicables:** dependiendo del país y de si el sistema se considera un instrumento de pago, dinero electrónico, o un servicio de custodia de fondos de terceros — esto varía mucho por jurisdicción y debe validarse legalmente antes del lanzamiento.
- **Auditoría y logs:** conservar el historial completo de depósitos, saldos y movimientos por el tiempo que exija la normativa local aplicable.

7. Resumen del stack técnico

Componente	Herramienta sugerida	Propósito
Derivación de wallets	tronweb / bip32 / bip39	Generar direcciones únicas desde una sola semilla
Generación de QR	librería qrcode (npm)	Representar la dirección visualmente
Lectura de blockchain	API TronGrid / nodo propio	Consultar transacciones entrantes
Notificaciones	Webhooks (TronGrid, Tatum)	Alternativa a polling para detectar depósitos
Custodia de fondos	Wallet fría + esquema multisig	Evitar control unilateral sobre fondos de terceros
Base de datos	Postgres / MySQL con transacciones	Persistir saldos y depósitos de forma atómica

8. Checklist antes de lanzar

- [] Semilla maestra generada y almacenada cifrada (KMS/HSM), nunca en texto plano.
- [] Direcciones derivadas correctamente probadas en testnet antes de producción.
- [] Sistema de detección de depósitos probado con reorgs simuladas / confirmaciones mínimas definidas.
- [] Idempotencia verificada (mismo tx_hash no acredita dos veces).

- ☐ Proceso de sweep hacia wallet fría / multisig probado y con logs auditables.
- ☐ Revisión legal de KYC/AML y licencias aplicables en la(s) jurisdicción(es) objetivo.
- ☐ Plan de respuesta ante incidentes (ej. clave comprometida, depósito duplicado, disputa de usuario).

Documento de referencia técnica general — no constituye asesoría legal ni financiera. Adaptar según la red blockchain, moneda y jurisdicción específicas del proyecto.